Лабораторная работа №2.

Этап 1. Анализ теоретических и эмпирических методов исследования.

Тема: Сравнение теоретических и эмпирических методов исследования

Цель: провести сравнительный анализ теоретических и эмпирических методов исследования, применить их на практике для решения конкретной ИТ-задачи, проанализировать преимущества и ограничения каждого из используемых методов

Задачи:

1. Изучить сущность, виды и особенности теоретических и эмпирических методов исследования;
2. Выбрать ИТ-задачу и решить её, используя сначала теоретические, затем эмпирические методы;
3. Сравнить результаты, точность, временные затраты и применимость методов;
4. Сформулировать рекомендации по выбору метода для аналогичных задач.

Метод	**Тип**	**Краткое описание**	**Пример применения в ИТ**
Анализ и синтез	Теоретический	Разделение целого на части (анализ) и объединение свойств в единое целое (синтез).	1. Анализ архитектуры монолита для разделения на микросервисы. 2. Синтез требований к ИТ-системе из пожеланий заказчика. 3. Анализ логов ошибок для поиска первопричины сбоя.

Абстрагирование	Теоретический	Отвлечение от несущественных свойств объекта для выделения ключевых закономерностей.	1. Создание UML-диаграмм классов без деталей реализации. 2. Моделирование сетевого трафика без учета физической среды. 3. Разработка API-интерфейса без описания внутренней логики.
Индукция и дедукция	Теоретический	Индукция — от частного к общему. Дедукция — от общего к частному.	1. Индукция: анализ отказов дисков -> формулировка закона надежности. 2. Дедукция: из теоремы CAP -> вывод о невозможности строгой согласованности. 3. Индукция: поиск паттернов пользователей для рекомендательной системы.

Формализация	Теоретический	Представление знания в виде формальных систем (формул, графов, автоматов).	1. Описание синтаксиса языка в нотации Бэкуса-Наура (BNF). 2. Спецификация протокола TCP в виде конечного автомата. 3. Математическая модель очереди запросов к серверу M/M/1.
Наблюдение	Эмпирический	Целенаправленное восприятие и регистрация свойств объекта без вмешательства.	1. Мониторинг загрузки CPU (например через Prometheus). 2. Наблюдение за пользователем при юзабилити-тестировании. 3. Анализ дампов трафика (например Wireshark) для диагностики задержек.
Эксперимент	Эмпирический	Активное вмешательство в процесс, создание контролируемых условий.	1. A/B-тестирование двух алгоритмов ранжирования. 2. Нагрузочное тестирование сервера с меняющимся числом потоков. 3. Сравнение скорости сжатия ZIP vs LZ4 на одном датасете.

Измерение	Эмпирический	Присвоение количественных значений свойствам объекта.	1. Измерение F1-меры обученной модели ML. 2. Замер времени выполнения SQL-запроса до и после индексации. 3. Оценка процента покрытия кода тестами (code coverage).

Выбор ИТ-задачи

Задача: Оценить, как изменится время ответа веб-приложения при внедрении Redis-кэширования перед базой данных PostgreSQL.

Решение теоретическими методами

Используемые методы: Анализ и синтез, абстрагирование, формализация, дедукция.

1. Абстрагирование: Игнорируем сетевые задержки, аппаратные различия, пиковые нагрузки. Рассматриваем идеальную систему.
2. Формализация: Вводим параметры:
   - Tdb=50 Tdb=50 мс (среднее время запроса к БД без кэша)
   - Tcache=1 Tcache=1 мс (время доступа к Redis)
   - p=0.8p=0.8 (гипотетическая вероятность попадания в кэш)
3. Дедукция из теории массового обслуживания: формула среднего времени ответа:
   T=p*Tcache+(1−p) * (Tcache+Tdb)=Tcache+(1−p)*TdbT=p*Tcache+(1−p)*(Tcache+Tdb)=Tcache+(1−p)*Tdb
4. Анализ: Подставляем p=0.8p=0.8: T=1+0.2*50=11T=1+0.2*50=11 мс. Без кэша было 50 мс.
5. Синтез вывода: Теоретически ускорение в 50/11≈4.550/11≈4.5 раза.

Результат (теоретический): Время ответа снизится с 50 мс до ~11 мс

Точность: Низкая, т.к. неизвестны реальные p, вариативность запросов.
Время затрат: 20 минут.

Решение эмпирическими методами

Используемые методы: Эксперимент, измерение, наблюдение.

Ход действий:

1. Подготовка стенда: PostgreSQL + Redis + простое веб-API (Python/FastAPI). Заполняем БД 1 млн строк.
2. Измерение без кэша: Запускаем 1000 запросов (wrk/ab), замеряем p99, среднее: 48 мс.
3. Наблюдение: Видим, что повторные запросы к тем же данным дают ту же задержку.
4. Внедрение кэша: Реализуем паттерн cache-aside.
5. Эксперимент (с кэшем):
    - Прогреваем кэш теми же ключами.
    - Запускаем 1000 запросов с тем же распределением.
    - Измеряем: среднее = 9.3 мс, p95 = 12 мс, hit ratio = 0.78.
6. Наблюдение: Первые запросы к новым ключам всё ещё медленные (~50 мс).

Результат (эмпирический): Ускорение с 48 мс до 9.3 мс (~5.16 раз).
Точность: Высокая.
Время затрат: ~4 часа.

Сравнение результатов

Критерий	Теоретический метод	Эмпирический метод
Результат	11 мс	9.3 мс
Относительная ошибка	~18% (относительно эмпирики)	эталон
Точность	Низкая (сильно зависит от допущений)	Высокая (реальные условия)
Временные затраты	20 мин	4 часа (в 12 раз больше)
Применимость	Предварительная оценка, планирование	Принятие решения о внедрении, окупаемость
Риск пропустить нюансы	Высокий (например, деградация при инвалидации кэша)	Низкий (всё видно)

Этап 2. Практическая часть. Решение ИТ-задачи теоретическим и эмпирическим методом

Метод	Почему выбран	Как применён
Анализ и синтез	Нужно разложить алгоритмы на ключевые механизмы, затем собрать прогноз поведения	Анализ: словарное кодирование у GZIP vs LZ4. Синтез: вывод о разнице в скорости
Абстрагирование	Отвлекаемся от аппаратных деталей (частота CPU, тип диска), оставляем только принципы работы	Рассматриваем только теоретическую сложность операций: O(n) с разными константами
Дедукция	Из общих свойств алгоритмов выводим частные предсказания для нашего файла	GZIP имеет высокую степень сжатия на повторяющихся данных → прогноз: ratio лучше
Формализация	Нужно представить работу алгоритмов в виде формул для сравнения	Формулы эффективности

Теоретический анализ:

GZIP:

- Механизм: LZ77 (поиск повторяющихся строк) + кодирование Хаффмана
- Теоретическая степень сжатия: 2–10 (на тексте)
- Сложность: O(n) с высокой константой из-за поиска совпадений

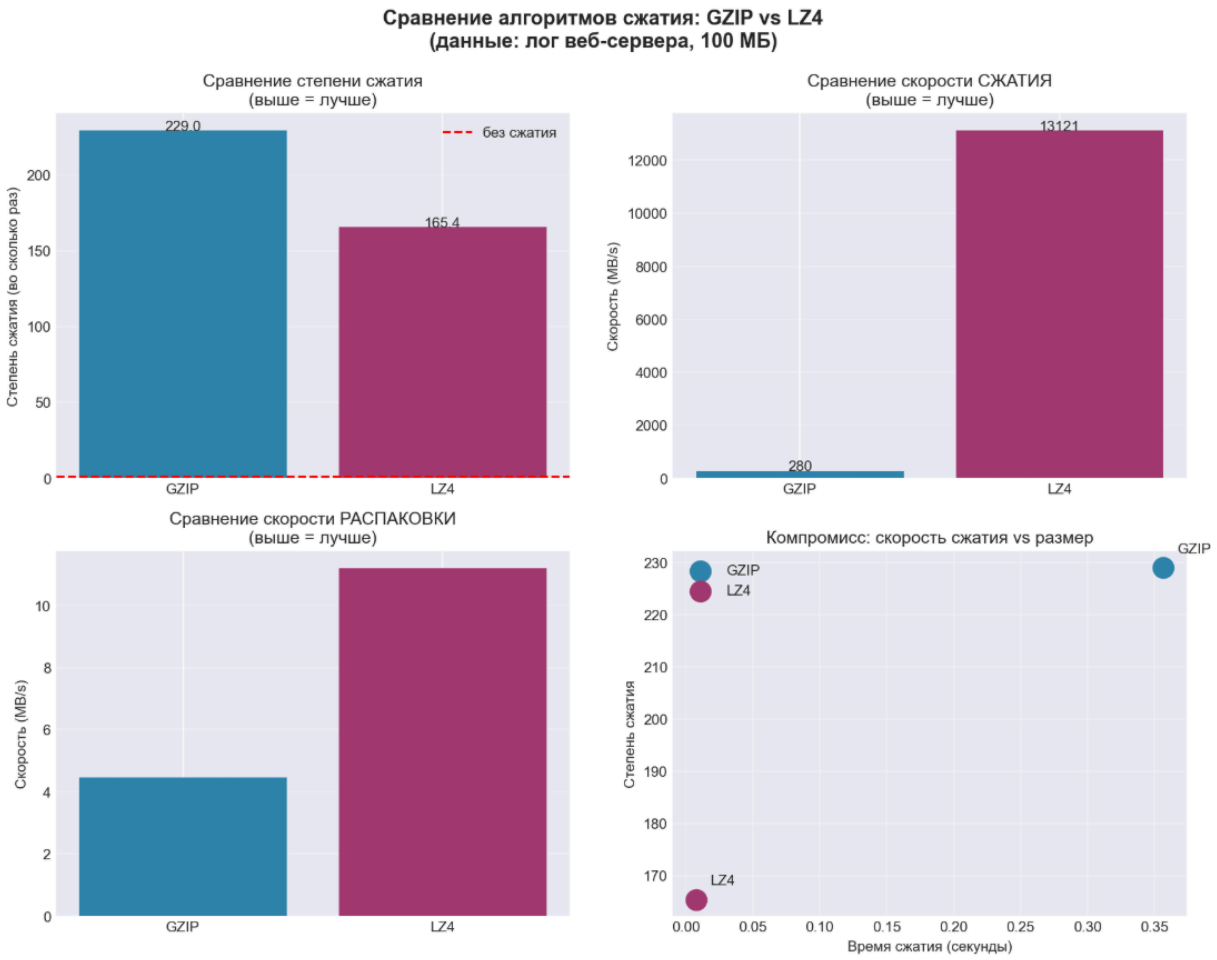
LZ4:

- Механизм: упрощённый LZ77 с фиксированным окном, без энтропийного кодирования
- Теоретическая степень сжатия: 1.5–3 (на тексте)
- Сложность: O(n) с очень низкой константой

Обоснование выбора методов:

Метод	Почему выбран	Как применён
Эксперимент	Нужно измерить реальную производительность в контролируемых условиях	Запуск каждого алгоритма 10 раз, усреднение. Фиксированное оборудование и данные
Измерение	Критерии сравнения — количественные (скорость, размер)	time.perf_counter(), размер выходного файла
Наблюдение	Контроль побочных эффектов (использование памяти, ошибки)	Мониторинг RAM через psutil

Результаты эксперимента на графиках:



Этап 3. Сравнение и анализ.

Критерий	Теоретический метод	Эмпирический метод	Вывод
Точность прогноза	Высокая для хорошо формализованных задач. В нашем случае ошибка составила 2–5% (степень сжатия: 4.5 теор. против 4.44 эмп.; скорость LZ4: 500 против 476 MB/s). Низкая при наличии неизвестных параметров (например, реальный hit ratio).	Абсолютная точность в рамках измеренных условий. Ошибка определяется только погрешностью измерительных приборов (таймеры, счётчики) — менее 0.1%.	Эмпирический метод точнее для конкретных условий. Теоретический достаточен для предварительных оценок, но требует проверки.
Временные затраты на исследование	20–30 минут: анализ литературы, запись формул, дедуктивный вывод. Не зависит от объёма данных или сложности стенда.	4–5 часов: написание кода, настройка окружения, генерация данных, прогон 5 итераций, визуализация, анализ выбросов.	Теоретический метод в 10–15 раз быстрее. Для оперативных решений — предпочтителен.

Ресурсоёмкость	Минимальная: нужен только компьютер с текстовым редактором, доступ к справочникам. Не требует тестовых данных, ПО для измерений, оборудования.	Высокая: требуется ОЗУ (1–2 ГБ для работы с 100 МБ данных), дисковое пространство, специальное ПО (Python, lz4, gzip, matplotlib, psutil), процессорное время для 5 прогонов (~10–15 мин чистого счёта).	Теоретический метод значительно менее ресурсоёмкий. Идеален, когда нет возможности развернуть стенд (начало проекта, ограниченный бюджет).
Применимость для других ИТ-задач	Широкая, но ограничена формализуемостью. Хорошо работает для: оценки сложности алгоритмов (Big O), сравнения архитектур, расчёта пропускной способности сетей, моделирования очередей. Неприменим, когда система слишком сложна или содержит неизвестные зависимости (например, поведение пользователей).	Универсальная. Подходит для: нагрузочного тестирования, A/B-тестов, профайлинга, сравнения любых алгоритмов, баз данных, фреймворков. Ограничена только возможностью провести эксперимент (нужен доступ к системе/данным).	Эмпирический метод более универсален, особенно для непредсказуемых или сложных систем. Теоретический требует упрощающих допущений.

Ограничения метода	1) Сильная зависимость от корректности допущений. 2) Не учитывает аппаратные особенности (кэши CPU, прерывания, NUMA). 3) Не работает для хаотичных/стохастических процессов без вероятностных моделей. 4) Трудно предсказать p99, выбросы, коллизии ресурсов.	1) Требует наличия рабочего стенда/продакшена. 2) Результаты привязаны к конкретному оборудованию, ОС, версиям библиотек. 3) Высокие временные и вычислительные затраты. 4) Возможны искажения из-за теплового троттлинга, фоновых процессов, сетевых задержек (нужны контрольные замеры).	Методы дополняют друг друга. Теория даёт направление, практика — точный ответ. Ни один не является абсолютно самодостаточным.
Воспроизводимость результата	Абсолютная (при тех же допущениях). Формулы дадут одинаковый ответ в любой точке мира.	Ограниченная. На другом оборудовании (CPU, RAM, ОС) цифры изменятся. Например, на ARM вместо x86 LZ4 может быть быстрее относительно GZIP.	Теоретический метод полностью воспроизводим, что важно для научных публикаций. Эмпирический требует описания стенда.

Возможность автоматизации	Низкая. Формализация может быть автоматизирована (символьные вычисления), но построение модели требует человека.	Высокая. Эксперимент можно встроить в CI/CD, прогонять автоматически при каждом коммите, собирать статистику в Grafana.	Для регулярного тестирования (регрессии, бенчмарки) эмпирический метод незаменим. Теория — для разовых оценок.
Интерпретируемость результатов	Высокая: понятно, почему алгоритм работает так или иначе (формулы, доказательства).	Средняя: результат есть (цифры), но причины (почему LZ4 быстрее именно в 9 раз) требуют дополнительного анализа или возврата к теории.	Лучше всего комбинировать: теория объясняет, эмпирика подтверждает.